

QUIZ GAME

OBJECT ORIENTED PROGRAMMING

Prepared by

Maria Nadarina, Ani Sukiasyan, Gayane Sahakyan

```
background: url(assets/img/bg.jpg);
background-size: 100vw 100vh;

}

.box{
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    width: 400px;
    padding: 40px;
    background: #000000;
    box-sizing: border-box;
    box-shadow: 0 15px 25px #000000;
    border-radius: 10px;
}

.box h2{
    margin: 0 0 30px;
    padding: 0;
    color: #ffff;
    text-align: center;
}

.box h3{
    margin: 0 0 10px;
    padding: 0;
    color: #ffff;
    text-align: center;
}

.box .inputBox{
    margin: 10px 0;
    padding: 10px;
    border: 1px solid #000000;
    border-radius: 10px;
    background: #000000;
    color: #ffff;
    width: 100%;
}
```


THIS PROJECT IS A QUIZ APPLICATION DEVELOPED USING OBJECT-ORIENTED PROGRAMMING PRINCIPLES IN JAVA.

THE PROGRAM SUPPORTS:

different quiz difficulties,
multiple question types,
score tracking,
exception handling,
console mode,
and graphical user interface mode.

```
background-size: 100% 100%;
position: absolute;
top: 50%;
left: 50%;
transform: translate(-50%, -50%);
width: 400px;
padding: 40px;
background: linear-gradient(to top right, transparent 49%, #000 49%, #000 51%, #fff 51%, #fff 53%);
box-sizing: border-box;
box-shadow: 0 15px 25px #000;
border-radius: 10px;
}
.box h2{
margin: 0 0 30px;
padding: 0;
color: #fff;
text-align: center;
}
.box h3{
margin: 0 0 10px;
padding: 0;
color: #fff;
text-align: center;
}
.box input{
```

THE MAIN GOAL OF THE PROJECT WAS TO APPLY CORE OOP CONCEPTS SUCH AS:

inheritance,
encapsulation,
abstraction,
polymorphism,
and modular design.

THE APPLICATION STARTS FROM THE QUIZGAME MAIN CLASS.

```
background-color: #1a1a1a;
background-size: 100% 100%;

position: absolute;
top: 50%;
left: 50%;
transform: translate(-50%, -50%);
width: 400px;
padding: 40px;
background: linear-gradient(to top right, transparent 49%, #333 49%, #333 51%, #f0f0f0 51%, #f0f0f0 53%);
box-sizing: border-box;
box-shadow: 0 15px 25px #333;
border-radius: 10px;
}

.box h2{
margin: 0 0 30px;
padding: 0;
color: #fff;
text-align: center;
}

.box h3{
margin: 0 0 10px;
padding: 0;
color: #fff;
text-align: center;
}

.box input{
```

PROJECT STRUCTURE



The project is divided into several packages:

model
questions
engine
factory
exceptions
console
gui

This structure makes the project organized and easy to maintain.

ESSENTIAL

THE MODEL PACKAGE CONTAINS:-

QUIZ
PLAYER
DIFFICULTYLEVEL

THE QUESTIONS PACKAGE CONTAINS:-

ABSTRACT CLASS QUESTION
MULTIPLECHOICEQUESTION
FILLINBLANKQUESTION
TRUEFALSEQUESTION
CODECOMPLETIONQUESTION

THE ENGINE PACKAGE CONTROLS THE QUIZ LOGIC USING:-

QUIZSESSION
ANSWERRESULT

THE FACTORY PACKAGE CREATES QUIZZES DYNAMICALLY USING QUIZFACTORY.

THE EXCEPTIONS PACKAGE HANDLES INVALID SITUATIONS SUCH AS:-

EMPTY ANSWERS,
INVALID DIFFICULTY,
INVALID SCORE,
AND FINISHED QUIZZES.

THIS MODULAR ARCHITECTURE DEMONSTRATES GOOD SOFTWARE ENGINEERING PRACTICES.

```
position: absolute;
top: 50%;
left: 50%;
transform: translate(-50%,
width: 400px;
padding: 40px;
background: rgba(0, 0, 0,
box-sizing: border-box;
box-shadow: 0 15px 25px rgba(0, 0, 0, 0.5);
border-radius: 10px;
}

.box h2{
margin: 0 0 30px;
padding: 0;
color: #fff;
text-align: center;
}

.box h3{
margin: 0 0 10px;
padding: 0;
color: #fff;
text-align: center;
}
```

MAIN CLASS AND PROGRAM FLOW

The program begins in the QuizGame class. The main method checks whether GUI mode is available.

If GUI is supported:
the graphical interface launches.

Otherwise:
the application automatically switches to console mode.

The console version is controlled
by **ConsoleQuizRunner**.

The flow of the application is:

1. User enters name
2. User selects difficulty
3. QuizFactory creates quiz
4. QuizSession starts
5. Questions are displayed
6. Answers are checked
7. Score is calculated
8. Final result is shown

The screenshot shows a web-based quiz interface titled "Object-Oriented Programming Quiz". The page has a light blue header with the title. Below the header, there is a welcome message: "Welcome to the Java OOP quiz game. This project demonstrates inheritance, abstraction, encapsulation, and polymorphism. Enter your name to begin." The main content area is white. At the bottom, there is a form with two sections. The first section is labeled "Player name:" and has a text input field containing the name "Gayane". The second section is labeled "Difficulty level:" and has a dropdown menu with four options: "Easy", "Medium", "Hard", and "Extremely Hard". The "Easy" option is selected and highlighted with a blue background. To the right of the dropdown menu, there is a yellow button with the text "Start Quiz".

OOP PRINCIPLES

1. ABSTRACTION

The abstract class Question defines common behavior for all question types.

It contains abstract methods such as:

checkAnswer()
getCorrectAnswer()
getAnswerType()

Each subclass provides its own implementation.

2. INHERITANCE

Classes like:

MultipleChoiceQuestion

FillInBlankQuestion

TrueFalseQuestion

inherit from the Question class.

This reduces duplicated code.



```
background: url(0);
background-size: 100vw 100vh;

}

.box{
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    width: 400px;
    padding: 40px;
    background: rgba(0, 0, 0, 0.5);
    box-sizing: border-box;
    box-shadow: 0 15px 25px rgba(0, 0, 0, 0.5);
    border-radius: 10px;
}

.box h2{
    margin: 0 0 30px;
    padding: 0;
    color: #fff;
    text-align: center;
}

.box h3{
    margin: 0 0 10px;
    padding: 0;
    color: #fff;
    text-align: center;
}
```

OOP PRINCIPLES

3. POLYMORPHISM

The program stores all question types inside a List<Question>. Even though objects are different subclasses, Java treats them through the parent type Question.

Different versions of checkAnswer() execute dynamically depending on object type.

4. ENCAPSULATION

The fields inside classes are private.

For example, in the Player class:

- name
- score

are private and accessed through getters and methods.

This protects internal data from incorrect modification.



```
background-size: 100% 100%;
}
.box{
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    width: 400px;
    padding: 40px;
    background: linear-gradient(to top right, transparent 49%, #000 49%, #000 51%, #fff 51%, #fff 52%, transparent 52%);
    box-sizing: border-box;
    box-shadow: 0 15px 25px #000;
    border-radius: 10px;
}
.box h2{
    margin: 0 0 30px;
    padding: 0;
    color: #fff;
    text-align: center;
}
.box h3{
    margin: 0 0 10px;
    padding: 0;
    color: #fff;
    text-align: center;
}
```


QUESTION SYSTEM

The question system is one of the most important parts of the project.

The application supports different question types:

multiple choice,
true/false,
fill in the blank,
code completion.

Each question type has:

its own answer validation,
its own behavior,
and scoring.

For example:

MultipleChoiceQuestion checks answers using letters like A, B, C, D.

TrueFalseQuestion accepts values like True, False, A, or B.

FillInBlankQuestion checks text answers.

The QuizFactory dynamically generates questions and even random programming and number system questions.

This makes every quiz different and more interactive.

QUIZ SESSION AND SCORING

The QuizSession class controls the quiz process.

Its responsibilities include:

- tracking current question,
- checking if quiz is finished,
- validating answers,
- updating scores,
- returning progress summaries.

When a user submits an answer:

- The system checks correctness
- Points are calculated
- Score is updated
- Next question is loaded

The score system depends on difficulty level.

The DifficultyLevel enum defines:

- Easy
- Medium
- Hard
- Extremely Hard

Each level has:

- different base points,
- different question count.

The final score is displayed as:

- points earned,
- total possible points,
- percentage.

EXCEPTION HANDLING AND GUI

The project also demonstrates exception handling.

Custom exceptions include:

- **EmptyAnswerException**
- **InvalidPlayerNameException**
- **InvalidDifficultyException**
- **QuizFinishedException**
- **InvalidScoreException**

For example:

if the player leaves the answer empty,
the system throws EmptyAnswerException.

This improves reliability and prevents invalid behavior.

The project also includes a graphical interface using Java Swing.

The GUI contains:

- **welcome screen,**
- **difficulty selection,**
- **progress bar,**
- **question display,**
- **buttons,**
- **and score updates.**

CONCLUSION

In conclusion, this project successfully demonstrates:

- Object-Oriented Programming concepts,
- Java GUI development,
- exception handling,

Through this project we improved our understanding of:

- inheritance,
- polymorphism,
- abstraction,
- encapsulation,
- and Java application architecture.

Future improvements could include:

- **database support,**
- **timer system,**
- **multiplayer mode,**
- **leaderboard,**
- **and file saving functionality.**



THANK YOU



```
background: url(0);
background-size: 100vw 100vh;

}

.box{
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  width: 400px;
  padding: 40px;
  background: rgba(0, 0, 0, 0.5);
  box-sizing: border-box;
  box-shadow: 0 15px 30px rgba(0, 0, 0, 0.5);
  border-radius: 10px;
}

.box h2{
  margin: 0 0 30px 0;
  padding: 0;
  color: #fff;
  text-align: center;
}

.box h3{
  margin: 0 0 10px 0;
  padding: 0;
  color: #fff;
  text-align: center;
}

.box .input{
```